

A partir del conjunto de datos definitivo utilizado para la construcción del modelo, se elaboraron dos matrices de correlación con el propósito de analizar las relaciones entre las variables independientes y la variable objetivo, especialmente frente a la categoría Y_{alto} , que representa los casos de mayor riesgo.

La primera matriz incluyó únicamente las variables numéricas del conjunto depurado, junto con la variable Y_alto. Para su elaboración, las fechas fueron transformadas a valores numéricos y las variables con valores faltantes fueron imputadas con la mediana.

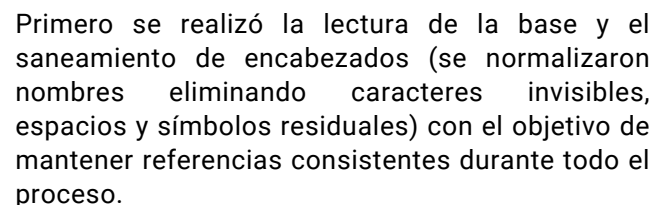
Matriz de correlación (Top numéricas vs Y_alto + Y_alto)

probacion
antigüedad_meses
AñoB
Cuotas
VALOR MEDIO
VALOR MAXIMO
VALOR BAJO
financiado
Mora
matricula
fianciacion
fecha de pago
cuota_inicial
NACIMIENTO
primera_cuota
Y_alto

1.00
0.75
0.50
0.25
0.00
-0.25
-0.50
-0.75
-1.00

ρ (Pearson)

En conjunto, ambos análisis brindan una primera aproximación a los factores que pueden influir en la clasificación de riesgo (aportan información útil para comprender el comportamiento de las variables y para construir modelos predictivos más precisos). Sobre esa base se aplicó una preparación común antes de entrenar cualquier algoritmo.



Luego se definió la variable objetivo y el conjunto de predictores (la variable de interés fue Y_categorica) y se excluyeron identificadores operativos que no aportan señal predictiva y pueden inducir fugas (Llave2, Nombre_linea, IDBANNER).

Modelos desarrollados y evaluación

A continuación se tipificaron las variables en numéricas y categóricas para habilitar un preprocesamiento diferenciado. En las numéricas se aplicó imputación por mediana; en las categóricas, imputación por la moda y codificación One-Hot con manejo de categorías poco frecuentes y desconocidas en inferencia (min_frequency cercano a 1 %). Todo el flujo se implementó con un ColumnTransformer (garantiza consistencia, eficiencia y uso de matrices esparsas cuando corresponde).

```
# Normaliza encabezados
df = df.copy()
df.columns = (
    df.columns.astype(str)
    .str.replace('\u00ff', '', regex=False)
    .str.strip()
)

# 1) Separar Y y X
TARGET = "Y_categorica"
DROP = ["Llave2", "Nombre_linea", "IDBANNER"]
y = df[TARGET].copy()
X = df.drop([TARGET] + [c for c in DROP if c in df.columns], axis=1).copy()

# 2) Columnas numéricas / categóricas
num_cols = [c for c in X.columns if pd.api.types.is_numeric_dtype(X[c])]
cat_cols = [c for c in X.columns if c not in num_cols]

# 3) Preprocesamiento
num_tf = SimpleImputer(strategy="median")
cat_tf = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(
        handle_unknown="ignore",
        sparse_output=True,
        min_frequency=0.01,
        dtype=np.float32
    ))
])

preprocess = ColumnTransformer(
    transformers=[
        ("num", num_tf, num_cols),
        ("cat", cat_tf, cat_cols),
    ],
    remainder="drop",
    sparse_threshold=1.0
)

# 4) Partición
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Validación del preprocesador
_ = preprocess.fit(X_train, y_train)

print("X:", X.shape, " | y:", y.shape)
print("Numéricas:", len(num_cols), " | Categóricas:", len(cat_cols))
print("Train:", X_train.shape, "Test:", X_test.shape)
```

X: (72292, 40) | y: (72292,)
Numéricas: 10 | Categóricas: 30
Train: (57833, 40) Test: (14459, 40)

La evaluación se estructuró con una partición estratificada 80/20 entre entrenamiento y prueba (el preprocesador se ajustó solo con entrenamiento y se validó su funcionamiento antes del modelado). Dado que se trata de un problema multiclase de morosidad, se priorizó el AUC macro One-Vs-Rest en validación cruzada y prueba, complementado con exactitud, precisión macro y recall macro (métricas sensibles al desbalance).

Esta configuración permite controlar la heterogeneidad y el desbalance, evitar fugas de información y mantener pipelines reproducibles para la experimentación y el despliegue.

Los hallazgos de correlación guiaron la priorización de familias de variables (temporales, condiciones del crédito, montos y modalidades) y motivaron el uso de métodos de ensamble basados en árboles (son adecuados para relaciones no lineales e interacciones, son robustos ante escalas heterogéneas y valores faltantes y producen probabilidades útiles para segmentación por riesgo).

La combinación de bosques aleatorios (bagging) y boosting (XGBoost y LightGBM) permite contrastar sesgo y varianza; además, los esquemas de selección de características como PSA y Boruta contribuyen a simplificar y estabilizar los modelos (favorecen la interpretabilidad y el mantenimiento en un contexto de negocio).

Se desarrollaron cinco modelos predictivos de morosidad, cada uno en un notebook independiente, con el propósito de visualizar y comparar su desempeño bajo distintos enfoques de modelado y selección de variables.

En todos los casos se aplicaron esquemas de búsqueda aleatoria de hiperparámetros (RandomizedSearchCV) y validación cruzada estratificada, optimizando el AUC macro bajo el enfoque One-Vs-Rest. De esta manera se buscó determinar cuál de los modelos ofrecía el mejor equilibrio entre capacidad discriminativa, estabilidad y generalización.

Modelo 1. Random Forest con RandomizedSearchCV

El primer modelo correspondió a un bosque aleatorio entrenado con ponderación balanceada para controlar el desbalance de clases. En el pipeline se incluyó imputación y codificación ordinal de variables categóricas. Se utilizó validación cruzada estratificada con tres particiones, optimizando el AUC macro One-Vs-Rest.

Los resultados entregaron el mejor conjunto de hiperparámetros, las métricas en la muestra de prueba, un reporte detallado por clase y las probabilidades de pertenencia a cada categoría de riesgo. Este algoritmo se seleccionó como línea base por su estabilidad frente al ruido y su capacidad para proporcionar medidas de importancia de variables, útiles para interpretación y lectura gerencial.

Modelo 2. XGBoost con RandomizedSearchCV

El segundo modelo se basó en XGBoost, un método de boosting de gradiente aplicado con imputación y codificación ordinal. La búsqueda aleatoria exploró parámetros como profundidad de los árboles, tasa de aprendizaje, pesos mínimos por nodo, parámetro gamma y tasas de muestreo. La validación cruzada fue estratificada y se optimizó el AUC macro.

El modelo arrojó la mejor configuración de hiperparámetros y métricas en prueba. XGBoost fue elegido por su capacidad para manejar relaciones no lineales y estructuras complejas entre variables, logrando buena discriminación incluso en escenarios con cierto desbalance.

Modelo 3. LightGBM con RandomizedSearchCV

El tercer modelo empleó LightGBM bajo la misma estructura de preprocesamiento. Se exploraron hiperparámetros como número de hojas, profundidad máxima, muestras mínimas por nodo, tasa de aprendizaje y regularizaciones. También se utilizó validación cruzada estratificada optimizando el AUC macro.

LightGBM se destaca por su eficiencia al procesar grandes volúmenes de datos y categorías, con tiempos de entrenamiento reducidos y un buen equilibrio entre sesgo y varianza, lo que lo convierte en una alternativa ágil y precisa.

Modelo 4. PSA (selección por modelo) con competencia RF, XGB y LGBM

El cuarto modelo incorporó una fase de selección de características mediante la técnica PSA (SelectFromModel) utilizando LightGBM como estimador base. Este procedimiento retuvo únicamente las variables con importancia superior a la mediana.

A partir de ese subconjunto reducido, se entrenaron tres modelos (Random Forest, XGBoost y LightGBM), que fueron comparados en validación cruzada y prueba con base en el AUC macro One-Vs-Rest, además de precisión y recall.

Esta estrategia permitió reducir la dimensionalidad, mejorar la interpretabilidad y agilizar el proceso de entrenamiento, manteniendo un nivel competitivo de desempeño.

Modelo 5. Boruta con competencia RF, XGB y LGBM

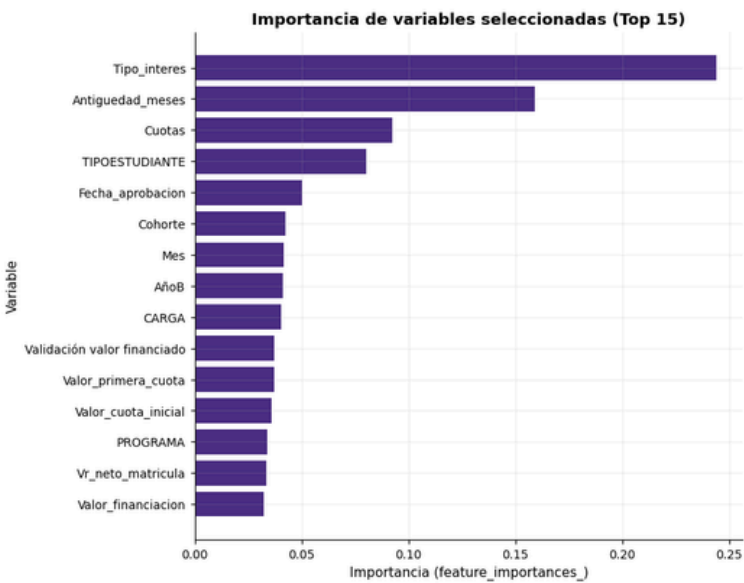
El quinto modelo utilizó el método Boruta para la selección de variables, tomando un Random Forest como estimador guía. Boruta identificó de manera estable las variables confirmadas como relevantes frente a permutaciones aleatorias.

Este subconjunto se aplicó luego a los pipelines finales mediante una transformación que preservó solo las columnas seleccionadas. Los modelos resultantes (BORUTA+RF, BORUTA+XGB y BORUTA+LGBM) se compararon según el AUC macro, exactitud, precisión y recall.

Boruta se destacó por su capacidad para identificar relevancia real de las variables y mitigar efectos de colinealidad, especialmente en bases con muchas variables categóricas.

Feature selection

El análisis de Boruta destacó variables coherentes con la lógica de morosidad crediticia.



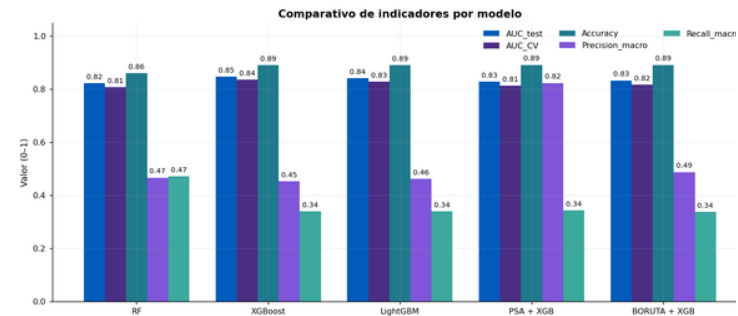
- Tipo_interes refleja la modalidad de cobro, cuyas condiciones afectan la capacidad de pago.
- Antigüedad_meses resume la trayectoria del crédito y su comportamiento a lo largo del tiempo.
- Cuotas define la exigencia mensual y el horizonte de exposición al riesgo.
- TIPOESTUDIANTE se relaciona con la permanencia académica, mientras que
- Fecha_aprobacion, Mes y AñoB incorporan el componente temporal y de cohortes.
- Cohorte y CARGA representan condiciones académicas y operativas que pueden incidir en la capacidad de cumplimiento.
- Variables como Valor_financiacion, Valor_primera_cuota, Valor_cuota_inicial y Validación valor financiado cuantifican el compromiso financiero, y
- PROGRAMA, FACULTAD, Vr_neto_matricula completan el perfil académico y socioeconómico del estudiante.

En conjunto, estas variables coinciden con los patrones observados en los análisis de correlación y respaldan la importancia de los factores temporales, las condiciones del crédito y las características académicas en la clasificación del riesgo.

Las metodologías PSA y Boruta, guiadas por estos hallazgos, permitieron concentrar el modelado en los factores de mayor capacidad explicativa, reduciendo ruido y redundancia y mejorando la estabilidad general de los modelos.

Evaluación de resultados y resumen de modelos

Modelo	Metodología	AUC_test	AUC_CV	Δ AUC (Test-CV)	Accuracy	Precision_macro	Recall_macro
RF	Random Forest (tuned)	0.82	0.81	0.01	0.86	0.47	0.47
XGBoost	XGB (tuned)	0.85	0.84	0.01	0.89	0.45	0.34
LightGBM	LGBM (tuned)	0.84	0.83	0.01	0.89	0.46	0.34
PSA + XGB	PCA/PSA + XGB	0.83	0.81	0.01	0.89	0.82	0.34
BORUTA + XGB	Boruta + XGB	0.83	0.82	0.02	0.89	0.49	0.34



A partir del comparativo, todos los enfoques muestran brechas pequeñas entre el AUC en validación cruzada y el AUC de prueba (aprox. 0.01–0.02), lo que sugiere modelos estables sin sobreajuste relevante. XGBoost presenta el mayor AUC_test (0.85), seguido por LightGBM (0.84) y Random Forest (0.82). En términos de exactitud, los métodos de boosting superan levemente a Random Forest (aprox. 0.89 frente a 0.86), y en precisión macro las diferencias son reducidas, entre 0.45 y 0.49, con excepción del pipeline PSA+XGB que destaca en precisión, aunque sin mejorar la capacidad de detección.

Bajo el objetivo del proyecto, identificar con oportunidad los casos de mayor riesgo, se prioriza el recall macro porque mide la capacidad del modelo para recuperar verdaderos positivos en todas las clases, especialmente en la categoría de riesgo alto. En este criterio, Random Forest obtiene el mejor resultado (0.47), superando de manera clara a XGBoost y LightGBM (0.34). Además, RF presenta la menor diferencia entre AUC en prueba y validación cruzada (aprox. 0.01), lo que refuerza su estabilidad fuera de muestra. En consecuencia, aunque XGBoost alcanza el AUC más alto, se selecciona Random Forest como modelo de referencia por ofrecer un mejor equilibrio entre capacidad discriminativa, estabilidad y mayor recall.

Este comportamiento indica que logra capturar de forma más efectiva las diferentes categorías, y en especial la de riesgo alto, que es la más relevante para la toma de decisiones en estrategias de cobranza y prevención. Además, Random Forest aporta interpretabilidad mediante la importancia de variables, facilitando la comprensión y el seguimiento del riesgo. Por último, los enfoques con selección de variables (PSA y Boruta) no mejoraron significativamente el desempeño global frente a las versiones ajustadas, por lo que se recomienda mantener RF como base y utilizar los resultados de selección de variables como apoyo explicativo y de gobernanza del modelo.

Luego de determinar que el modelo Random Forest representaba la mejor alternativa, se procedió a continuar el proceso de experimentación en una herramienta de seguimiento y gestión de modelos del tipo MLflow, en este caso Databricks. Allí se registraron los experimentos realizados, permitiendo documentar las métricas, los hiperparámetros y las versiones del modelo de manera estructurada y reproducible.

Databricks con MLflow

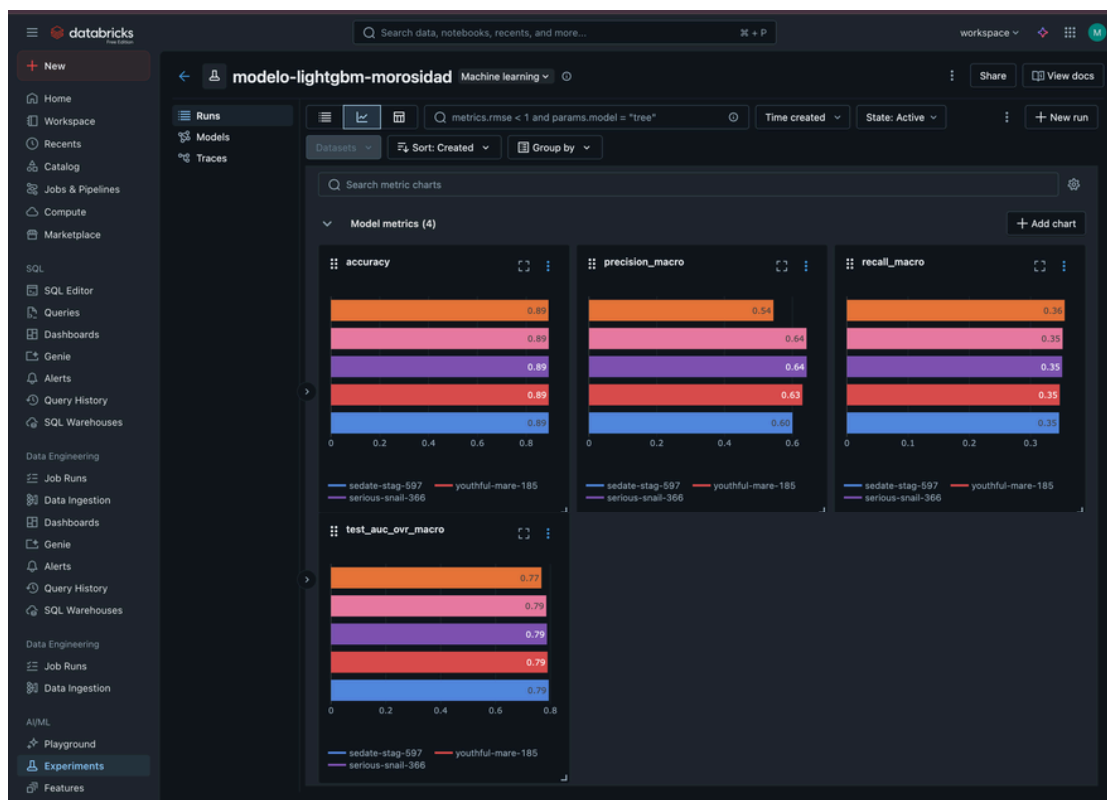
https://dbc-75a70282-d4ac.cloud.databricks.com/ml/experiments/4331751283863737/runs?o=3079839572844565&searchFilter=&orderByKey=attributes.start_time&orderByAsc=false&starTime=ALL&lifecycleFilter=Active&modelVersionFilter=All+Runs&datasetsFilter=W10%3D

Resultados del Modelo LightGBM

Todos los experimentos y ejecuciones de los modelos fueron desarrollados en Databricks, utilizando MLflow para el registro de métricas, seguimiento de experimentos y comparación de resultados. En este entorno se entrenó el modelo LightGBM (LGBM), cuyos resultados mostraron un desempeño moderado frente al problema de clasificación del riesgo crediticio.

El AUC promedio del modelo se ubicó entre 0.77 y 0.79, lo que evidencia una capacidad limitada para discriminar adecuadamente entre los niveles de riesgo. Aunque la exactitud (accuracy) alcanzó valores cercanos a 0.89, este indicador puede estar influenciado por el desbalance de clases, por lo que no necesariamente refleja un buen poder predictivo. Las métricas de desempeño por clase refuerzan esta lectura: el precision_macro osciló entre 0.54 y 0.64, mientras que el recall_macro se mantuvo alrededor de 0.35, indicando que LightGBM tuvo dificultades para identificar correctamente las clases minoritarias y capturar patrones relevantes en el dataset.

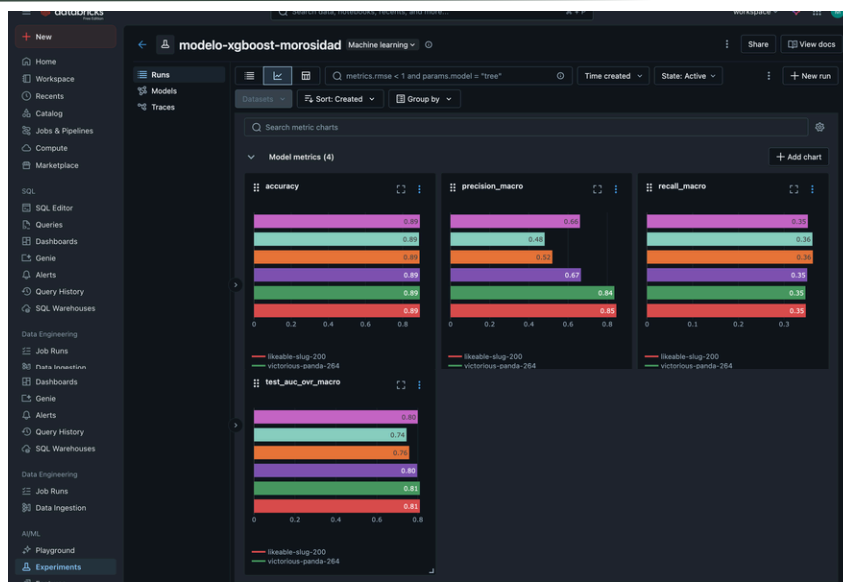
En conjunto, estos resultados posicionan a LGBM como el modelo con menor rendimiento entre los tres algoritmos evaluados, sirviendo como punto de comparación para los siguientes modelos: XGBoost y Random Forest.



Resultados del Modelo XGBoost

El modelo XGBoost también fue entrenado y evaluado dentro del entorno de Databricks, utilizando MLflow para registrar los experimentos y comparar las métricas entre ejecuciones. Este modelo mostró un desempeño superior al alcanzado por LightGBM, especialmente en términos de discriminación entre clases y métricas de precisión. El AUC promedio osciló entre 0.76 y 0.81, alcanzando picos de 0.81, lo que refleja una capacidad más robusta para separar correctamente los diferentes niveles de riesgo crediticio.

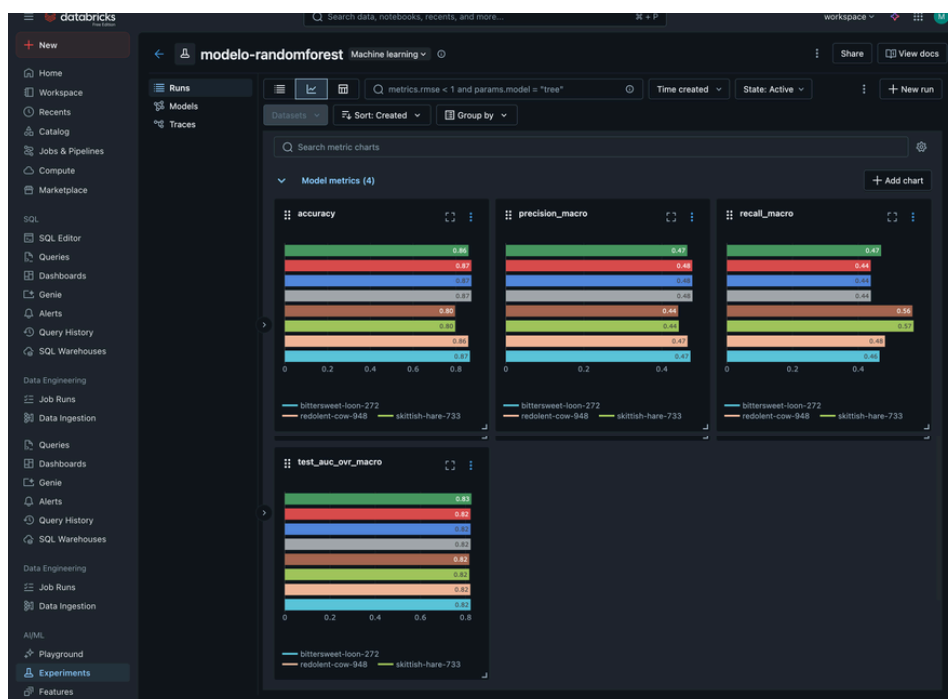
Si bien la exactitud (accuracy) se mantuvo igual a LightGBM (alrededor de 0.89), las mejoras más notables se evidenciaron en el precision_macro, donde XGBoost logró valores entre 0.66 y 0.85, superando ampliamente el desempeño del modelo anterior. Esto indica que XGBoost clasifica con mayor certeza cuando predice una clase, especialmente en las categorías menos representadas. Por otro lado, el recall_macro permaneció cercano a 0.35–0.36, lo cual confirma que, aunque XGBoost es más preciso, aún enfrenta desafíos para capturar casos positivos de clases minoritarias. En conjunto, estos resultados posicionan a XGBoost como un modelo intermedio sólido, con una mejora clara frente a LGBM pero todavía por debajo del desempeño final obtenido con Random Forest.



Resultados del Modelo Random Forest

El modelo Random Forest, entrenado y monitoreado en Databricks mediante MLflow, obtuvo el mejor desempeño entre los tres algoritmos evaluados. En términos de discriminación, alcanzó un AUC promedio entre 0.82 y 0.83, superando consistentemente los valores obtenidos con LGBM y XGBoost. Esta métrica evidencia una capacidad más robusta del modelo para diferenciar correctamente entre los distintos niveles de riesgo crediticio.

Si bien la exactitud (accuracy) se mantuvo en rangos similares a los otros modelos (entre 0.86 y 0.87), las mejoras más importantes se reflejaron en métricas asociadas al desempeño por clase. El precision_macro se ubicó entre 0.44 y 0.48, y aunque es ligeramente inferior al de XGBoost, el recall_macro mostró valores superiores (hasta 0.57 en algunas ejecuciones), convirtiendo a Random Forest en el modelo con mayor capacidad para detectar correctamente las clases minoritarias. Esta mejora en sensibilidad lo posiciona como el modelo más equilibrado en términos de precisión y recall, y por tanto, el más adecuado para un caso real de riesgo crediticio donde los falsos negativos son críticos.



Random Forest logró el mejor equilibrio general: alcanzó el AUC más alto, mantuvo un buen nivel de precisión y demostró la mejor capacidad para identificar correctamente las clases minoritarias mediante un recall superior. Este balance entre métricas lo convierte en el modelo más estable y confiable para un sistema de clasificación de riesgo crediticio, donde la correcta identificación de estudiantes con mayor probabilidad de incumplimiento es fundamental para la toma de decisiones institucionales.